



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Ćwiczenie: C14

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji.

Obsługa błędów wykonania aplikacji

Obsługa błędów to kluczowy aspekt programowania, który pozwala na kontrolowanie nieprzewidzianych sytuacji w trakcie działania aplikacji. Błędy mogą wynikać z różnych przyczyn, takich jak nieprawidłowe dane wejściowe, problemy z dostępem do zasobów (np. plików, baz danych), czy błędy logiczne w kodzie. W języku C# obsługa błędów jest realizowana za pomocą mechanizmu wyjątków (exceptions).

Wyjątki to obiekty, które reprezentują błędy występujące w trakcie wykonywania programu. Gdy wystąpi błąd, system "rzuci" wyjątek, który może zostać "przechwycony" i obsłużony przez programistę. Dzięki temu aplikacja może działać dalej, nawet w przypadku wystąpienia problemów, lub bezpiecznie zakończyć działanie, informując użytkownika o błędzie.

W C# do obsługi błędów używamy bloków try-catch-finally:

- try: Blok, w którym umieszczamy kod, który może spowodować błąd.
- catch: Blok, w którym przechwytyjemy i obsługujemy wyjątek.
- finally: Blok, który jest wykonywany niezależnie od tego, czy wystąpił błąd, czy nie. Często służy do zwalniania zasobów.

Przykład 1. Podstawowa obsługa wyjątków

```
using System;

namespace ObsługaBledow
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Podaj dwie liczby do dzielenia:");
```

```

try
{
    Console.WriteLine("Liczba 1: ");
    int liczba1 = int.Parse(Console.ReadLine()); // Może wystąpić błąd, jeśli
użytkownik poda nieprawidłowe dane

    Console.WriteLine("Liczba 2: ");
    int liczba2 = int.Parse(Console.ReadLine()); // Może wystąpić błąd, jeśli
użytkownik poda nieprawidłowe dane

    int wynik = liczba1 / liczba2; // Może wystąpić błąd dzielenia przez zero
    Console.WriteLine($"Wynik dzielenia: {wynik}");
}
catch (DivideByZeroException)
{
    Console.WriteLine("Błąd: Nie można dzielić przez zero!");
}
catch (FormatException)
{
    Console.WriteLine("Błąd: Podano nieprawidłowe dane. Wprowadź liczby
całkowite.");
}
catch (Exception ex) // Ogólna obsługa innych wyjątków
{
    Console.WriteLine($"Wystąpił nieoczekiwany błąd: {ex.Message}");
}
finally
{
    Console.WriteLine("Blok finally: Ten kod zawsze się wykona, niezależnie od
błędów.");
}
}
}

```

Wyjaśnienie:

1. try. W tym bloku umieszczamy kod, który może spowodować błąd (np. dzielenie przez zero lub nieprawidłowe dane wejściowe).
2. catch. Przechwytyjemy konkretne wyjątki:
 - o DivideByZeroException: Gdy użytkownik próbuje dzielić przez zero.
 - o FormatException: Gdy użytkownik poda nieprawidłowe dane (np. tekst zamiast liczby).
 - o Exception: Ogólna obsługa innych wyjątków.
3. finally. Ten blok jest zawsze wykonywany, niezależnie od tego, czy wystąpił błąd.

Przykład 2. Obsługa błędów przy pracy z plikami

```

using System;
using System.IO;

namespace ObslugaBledowPlikow
{
    class Program
    {
        static void Main(string[] args)
        {
            string sciezka = "testowy_plik.txt";

            try
            {
                string zawartosc = File.ReadAllText(sciezka); // Próba odczytu pliku
                Console.WriteLine("Zawartość pliku:");
                Console.WriteLine(zawartosc);
            }
            catch (FileNotFoundException)
            {
                Console.WriteLine("Błąd: Plik nie istnieje!");
            }
        }
    }
}

```

```

    }
    catch (UnauthorizedAccessException)
    {
        Console.WriteLine("Błąd: Brak uprawnień do odczytu pliku!");
    }
    catch (Exception ex)
    {
        Console.WriteLine($"Wystąpił nieoczekiwany błąd: {ex.Message}");
    }
    finally
    {
        Console.WriteLine("Operacja zakończona.");
    }
}
}
}

```

Wyjaśnienie:

1. File.ReadAllText. Próba odczytu pliku, która może zakończyć się błędem, jeśli plik nie istnieje lub brakuje uprawnień.
2. FileNotFoundException. Wyjątek występujący, gdy plik nie istnieje.
3. UnauthorizedAccessException. Wyjątek występujący, gdy brakuje uprawnień do odczytu pliku.

Zadania do samodzielnego rozwiązania

1. Rozszerz przykład 1. Dodaj obsługę błędów dla sytuacji, gdy użytkownik poda liczbę większą niż int.MaxValue.
2. Rozszerz przykład 2. Dodaj możliwość tworzenia pliku, jeśli plik nie istnieje, zamiast wyświetlania komunikatu o błędzie.
3. Stwórz własny wyjątek. Zdefiniuj własny wyjątek NieprawidlowaTemperaturaException, który zostanie rzucony, gdy użytkownik poda temperaturę poniżej -273°C (zero absolutne).
4. Obsługa błędów w aplikacji bankowej. Stwórz prostą aplikację symulującą przelew bankowy. Obsłuż sytuacje, gdy użytkownik próbuje przełać więcej środków niż ma na koncie.

... po godzinach

Kod Zagłady Rozdział 2. Pułapka w kodzie

Marek siedział przy biurku, wpatrując się w ekran laptopa. W głowie miał mętlik. Ktoś celowo wprowadził do kodu nieznaną klasę wyjątku, a tajemniczy rozmówca ostrzegał go przed czymś większym. Musiał działać ostrożnie.

Postanowił przeanalizować kod jeszcze raz, tym razem z większą uwagą. Otworzył plik z definicją klasy NieoczekiwanyBładException i zaczął przeglądać każdą linijkę. Na pierwszy rzut oka wszystko wyglądało normalnie, ale Marek wiedział, że diabeł tkwi w szczegółach.

Nagle zauważył coś, co wcześniej umknęło jego uwadze. W konstruktorze klasy znajdował się fragment kodu, który wyglądał podejrzanie:

```
public NieoczekiwanyBladException(String message) {  
  
    super(message);  
  
    logError(message);  
  
    executePayload();  
  
}
```

Metoda `executePayload()` była kluczem. Marek otworzył jej definicję i zamarł. Kod w środku był skomplikowany, ale jedno było pewne - to nie była zwykła obsługa błędów. Metoda ta uruchamiała zewnętrzny skrypt, który mógł wykonywać dowolne operacje na systemie.

— To jest pułapka — szepnął do siebie Marek. — Ktoś chce wykorzystać ten kod do przejęcia kontroli nad systemem.

Musiał działać szybko. Wiedział, że nie może tego zrobić sam. Potrzebował pomocy, ale komu mógł zaufać? Kasia była jedyną osobą, która przyszła mu na myśl. Była doświadczoną programistką i zawsze mogli na siebie liczyć.

— Kasia, muszę ci coś pokazać — powiedział, podchodząc do jej biurka.

Kasia spojrzała na niego zaskoczona, ale widząc jego poważną minę, wiedziała, że to coś ważnego.

— Co się stało? — zapytała, odkładając laptopa.

Marek szybko wyjaśnił jej sytuację, pokazując podejrzany kod. Kasia zmarszczyła brwi, analizując linijki.

— To wygląda na celowe działanie — powiedziała w końcu. — Ktoś chce wykorzystać nasz system do własnych celów. Musimy to zgłosić.

— Ale komu? — zapytał Marek. — Nie wiemy, kto za tym stoi. Może to ktoś z naszego zespołu.

Kasia zastanowiła się przez chwilę, a potem powiedziała:

— Musimy zabezpieczyć kod i znaleźć sposób na śledzenie, kto wprowadzał te zmiany. Może uda nam się namierzyć sprawcę.

Marek skinął głową. Wiedział, że to będzie trudne, ale nie mieli wyboru. Musieli działać szybko i ostrożnie.

— Zaczniemy od zabezpieczenia kodu — powiedział. — Potem spróbujemy namierzyć, kto za tym stoi.

Kasia zgodziła się i oboje wrócili do swoich komputerów. Wiedzieli, że to dopiero początek, a przed nimi długa droga do odkrycia prawdy. Ale byli zdeterminowani, by znaleźć odpowiedzi i powstrzymać zagrożenie, zanim będzie za późno.

Czy Marek i Kasia zdołają zabezpieczyć kod i namierzyć sprawcę? Jakie kolejne pułapki czekają na nich w systemie? Tego dowiemy się w kolejnych rozdziałach...